Research Square

Preprints are preliminary reports that have not undergone peer review.
They should not be considered conclusive, used to inform clinical practice,
or referenced by the media as validated information.

Containment over Detection: Cryptographic Boundary Enforcement for Prompt Injection Defense in Agentic LLM Systems

Saurabh Sharma

saurabh0880@gmail.com

Amazon

Research Article

Keywords: Prompt Injection, LLM Security, Agentic Systems, Containment Defense, Property-Based Testing, Adversarial Robustness

Posted Date: July 1st, 2026

DOI: https://doi.org/10.21203/rs.3.rs-10196595/v1

License: This work is licensed under a Creative Commons Attribution 4.0 International License.
Read Full License

Additional Declarations: The authors declare no competing interests.

Containment over Detection: Cryptographic Boundary Enforcement for Prompt Injection Defense in Agentic LLM Systems

Saurabh Sharma, Ramani Rajendran, Sameeraja KL, Shivansh Malhotra, Neelesh Dubey, and Yogendra Hada

Independent Research

Abstract

Prompt injection—the manipulation of an LLM-based agent through adversarial content embedded in user input—poses a critical security risk in production agentic systems with access to sensitive datastores and code execution environments. Detection-based defenses (keyword filtering, learned classifiers) suffer from a fundamental accuracy–latency tradeoff and fail to address the root cause: the absence of a syntactic boundary between instructions and data in LLM context windows. We present a *containment-based* defense architecture that shifts the security objective from detecting malicious input to rendering injection attempts structurally inert. The defense comprises four composable layers: (1) ephemeral cryptographic token generation providing 2^{256} bits of boundary entropy, (2) input canonicalization eliminating encoding-based bypass vectors, (3) token-authenticated XML envelope wrapping with server-side escape validation, and (4) behavioral prompt grounding that leverages the envelope as a trust signal. We formalize the security guarantee and prove that, under a black-box attacker model without oracle access to the runtime token, the probability of crafting a valid boundary escape is negligible ($\leq 2^{-256}$). The architecture is validated through 15 machine-checkable correctness properties using the Hypothesis property-based testing framework—three of which caught security-critical bugs before production deployment. Empirical evaluation against 547 adversarial patterns drawn from four established attack corpora (Garak, OWASP LLM Top 10, HarmBench prompt injection subset, and a custom multilingual probe set) demonstrates a 94.3% syntactic containment rate; the 5.7% residual failures are confirmed to be model-level semantic jailbreaks orthogonal to input-layer defense. Measured runtime overhead is <0.5ms per request. The evaluation harness and property specifications are released as open-source.[1]

Keywords: prompt injection, LLM security, agentic systems, containment defense, property-based testing, adversarial robustness

1 Introduction

Large Language Model (LLM) agents deployed in production environments increasingly operate with access to sensitive datastores, code execution sandboxes, and external APIs. These *agentic systems*—where the LLM autonomously selects tools, queries databases, and generates executable code—face a class of vulnerability known as *prompt injection* [Perez and Ribeiro, 2022]: adversarial instructions embedded in user input that attempt to override system-level behavioral constraints, exfiltrate data, or escalate privileges.

The attack surface is particularly acute in systems that ingest semi-structured natural language from collaborative channels (e.g., enterprise chat, ticketing systems, email), where users

[1] `https://github.com/saurabh0880/token-containment-defense`

routinely paste external content—forwarded tickets, diagnostic logs, code snippets—that may contain embedded adversarial instructions either intentionally (direct injection) or through compromised upstream sources (indirect injection) [Greshake et al., 2023].

We report on the design, formal analysis, and empirical validation of a containment-based defense deployed in a production agentic system serving ∼200 analysts. The system processes natural language queries with access to 16 production database tables, cloud object storage containing analytical data lakes, a sandboxed Python code interpreter, and a retrieval-augmented generation (RAG) knowledge base. A successful injection in this context could exfiltrate sensitive operational metrics, corrupt analytical reports, or attempt sandbox escape.

1.1 The Detection Trap

The dominant paradigm for prompt injection defense relies on *detection*: identifying malicious input before it reaches the LLM via keyword matching, learned classifiers, or perplexity-based anomaly detection [Alon and Kamfonas, 2023, Jain et al., 2023]. We argue this paradigm suffers from a fundamental limitation: *detection solves the wrong problem.* Even a 99% accurate detector passes 1% of attacks, and the attacker-defender asymmetry means the attacker needs only one success. Moreover, detection approaches face an inherent accuracy–latency–maintenance tradeoff: high-accuracy classifiers introduce inference latency and require continuous retraining as attack techniques evolve.

The correct security objective is not to detect injection attempts but to *render them structurally inert*—to ensure that adversarial content, even if present, cannot escape its designated data boundary and influence system behavior. This is the principle of *containment over detection.*

1.2 Contributions

This paper makes the following contributions:

1. A formal characterization of the containment security guarantee, proving negligible escape probability under black-box attacker assumptions (Theorem 1).

2. A four-layer composable defense architecture with formally specified interfaces and failure-mode isolation between layers.

3. A property-based testing methodology yielding 15 machine-checkable security invariants, three of which caught production-critical bugs missed by conventional unit testing.

4. Empirical evaluation against 547 adversarial patterns from four attack corpora, with controlled comparison against three baseline defenses: keyword filtering, learned classification, and prompt-level sandwich defense.

2 Background and Related Work

2.1 Prompt Injection Taxonomy

Prompt injection was first characterized by Perez and Ribeiro [2022] as the LLM analogue of SQL injection: user-supplied data interpreted as instructions due to the absence of a syntactic boundary in the LLM context window. Greshake et al. [2023] extended the taxonomy to *indirect* injection, where adversarial payloads are embedded in documents, web pages, or tool outputs retrieved by the agent. Liu et al. [2023] provide a comprehensive survey of attack types, categorizing them along axes of directness (direct vs. indirect), persistence (ephemeral vs. stored), and target (exfiltration, manipulation, denial of service).

Recent work has demonstrated increasingly sophisticated attack vectors: Zou et al. [2023] showed that adversarial suffixes can be automatically optimized to bypass safety training; Yi et al. [2024] demonstrated that indirect injection through retrieval is especially difficult to defend against; and Pasquini et al. [2024] showed that neural-network-generated adversarial prompts can evade both perplexity filters and learned classifiers.

2.2 Existing Defense Approaches

Detection-based defenses. Alon and Kamfonas [2023] train classifiers on injection examples but report precision degradation as attackers adapt. Jain et al. [2023] evaluate perplexity-based detection but find it ineffective against semantically fluent injections. Rebuff [Rebuff, 2023] combines heuristic and LLM-based detection layers but introduces significant latency overhead.

Prompt-level defenses. NeMo Guardrails [Rebedea et al., 2023] uses dialogue management to constrain LLM outputs. The sandwich defense [Sandwich Defense, 2023] repeats system instructions after user input. OpenAI [2024] propose an instruction hierarchy where the model is trained to prioritize system-level instructions over user-level content.

Architectural defenses. Spotlighting [Hines et al., 2024] uses token-level markers to distinguish data from instructions. StruQ [Chen et al., 2024] proposes structured queries that separate instruction and data channels at the prompt level. Wallace et al. [2024] formalize dual-LLM architectures where a privileged model validates the outputs of an unprivileged model operating on untrusted input.

Our positioning. The containment approach is complementary to prompt-level and model-level defenses. It operates at the *input preprocessing layer* and provides a deterministic syntactic guarantee that is independent of model behavior. Unlike detection-based approaches, it does not reject input—it neutralizes it. Unlike Spotlighting, it uses a cryptographically unguessable boundary token rather than a fixed delimiter, eliminating delimiter-stuffing attacks.

2.3 Property-Based Testing for Security

Property-based testing [MacIver et al., 2019] generates inputs from declared strategies and checks universal invariants, enabling discovery of edge cases that example-based tests miss. Claessen and Hughes [2000] established the paradigm; the Hypothesis framework [MacIver et al., 2019] extends it with stateful testing, database-backed example storage, and health checks. We adopt property-based testing as the primary validation methodology for security invariants, complementing adversarial evaluation.

3 Threat Model

3.1 System Model

We consider an agentic LLM system $\mathcal{S}$ that processes natural language queries q and has access to a set of tools $\mathcal{T} = \{t_1, \ldots, t_k\}$ including database queries, file operations, and code execution. The system constructs a prompt $P = [s \| w(q)]$ where s is the system prompt and $w(q)$ is the wrapped user query.

3.2 Attacker Model

Definition 1 (Black-box Prompt Injection Attacker)**.** *An attacker $\mathcal{A}$ is a probabilistic polynomial-time (PPT) algorithm that:*

1. *Can submit arbitrary strings $q \in \Sigma^*$ as user input.*

2. *Has no oracle access to the runtime security token τ.*

3. *Has full knowledge of the defense architecture (Kerckhoffs's principle).*

4. *Cannot observe or modify the system prompt s, the wrapping function w, or the token τ at rest in memory.*

The attacker's objective is to craft input q^* such that the LLM executes instructions from q^* as if they were system-level commands—i.e., the LLM treats content within the data envelope as instructions.

3.3 Trust Boundaries

- **Untrusted:** Raw user input q, retrieved documents from RAG, tool outputs from sandboxed execution.
- **Trusted:** Container runtime memory (holds τ), system prompt s, wrapping logic w, server-side validation.

3.4 Security Goal

Definition 2 (Containment)**.** *A defense provides containment if, for all attacker-chosen inputs $q^* \in \Sigma^*$, the wrapped output $w(q^*)$ preserves the structural integrity of the envelope—i.e., q^* cannot syntactically close the envelope tag or inject content outside the designated data boundary.*

4 Failure Analysis of Detection-Based Defenses

Before presenting the containment architecture, we characterize the failure modes of detection-based approaches observed during initial deployment.

4.1 Keyword Filtering

An initial defense used 51 injection-indicative patterns (e.g., "ignore previous," "you are now," "system prompt"). Evaluation on 500 production messages revealed:

- **False positive rate: 8.3%.** Analysts routinely write "ignore the previous week's data" or "you are now looking at the Q3 metrics"—legitimate analytical instructions matching injection heuristics.
- **Trivial bypass:** Unicode homoglyphs (e.g., Cyrillic characters substituted for Latin), zero-width joiners, and semantic paraphrasing ("disregard prior context" → "please set aside what came before") all evade keyword matching.
- **Maintenance burden:** The keyword list required weekly updates as new bypass techniques were reported.

4.2 Learned Classification

A distilled BERT-based classifier trained on 1,200 labeled messages reduced false positives to 2.1% but introduced 180ms p50 latency per request, required a separate model serving infrastructure, and demanded periodic retraining as the attack distribution shifted.

4.3 Sandwich Defense

Repeating system instructions after user input [Sandwich Defense, 2023] reduced successful injections but introduced prompt bloat (duplicating ∼2K tokens of system instructions), increased inference cost, and provided no formal guarantee—the model could still attend to injected instructions over repeated system instructions for sufficiently persuasive payloads.

4.4 Fundamental Limitation

All detection and prompt-repetition approaches share a structural deficiency: they operate probabilistically over natural language semantics. No semantic analysis can guarantee correctness because the decision boundary between "legitimate analytical instruction" and "injection attempt" is context-dependent and fundamentally ambiguous in natural language.

5 The Containment Architecture

The containment defense operates on a different principle: rather than classifying input as safe or unsafe, it *wraps all input* in a cryptographically authenticated envelope that the LLM is instructed to treat as data. The key insight is that this shifts the security property from semantic classification (undecidable in general) to syntactic containment (deterministically verifiable).

5.1 Layer 1: Ephemeral Token Generation

At container startup, a cryptographically secure random token τ is generated:

$$\tau \leftarrow \texttt{CSPRNG}(256) \text{ encoded as URL-safe base64 (43 characters)}$$

Properties of τ:

- Generated once per container lifecycle; never persisted to disk, logs, or environment variables.
- Entropy: 256 bits (collision probability $< 2^{-128}$ by birthday bound for 2^{64} containers).
- Rotation: automatic on container restart, providing forward secrecy against token compromise.

5.2 Layer 2: Input Canonicalization

Before wrapping, input is canonicalized to eliminate encoding-based bypass vectors:

```python
import re

def canonicalize(text: str) -> str:
    # Remove all Unicode control characters including
    # line/paragraph separators that behave as whitespace
    # in some tokenizer implementations
    CONTROL_CHARS = re.compile(
        r'[\x00-\x1F\x7F-\x9F\u2028\u2029]'
    )
    sanitized = CONTROL_CHARS.sub('', text)
    return " ".join(sanitized.split())
```

Listing 1: Input canonicalization

The canonicalization removes characters in Unicode categories Cc (control) and specific Zl/Zp (line/paragraph separators) that could be exploited to inject visual whitespace interpreted differently by the tokenizer versus the validation logic.

5.3 Layer 3: Token-Authenticated Envelope Wrapping

```python
def wrap(user_input: str, token: str) -> str:
    # Server-side validation: reject input containing
    # the token or envelope structure
    if token in user_input:
        raise SecurityViolation("Token present in input")
    envelope_open = f'<untrusted_input id="{token}">'
```

```
7     if envelope_open in user_input:
8         raise SecurityViolation("Envelope tag in input")
9
10    canonical = canonicalize(user_input)
11    return f'{envelope_open}{canonical}</untrusted_input>'
```

Listing 2: Envelope wrapping with validation

The validation in lines 4–7 is the critical security mechanism. Without it, an attacker could close the envelope tag by including `</untrusted_input>` in their input and inject instructions outside the boundary.

5.4 Layer 4: Behavioral Prompt Grounding

The system prompt includes instructions that leverage the envelope as a trust signal:

1. Content within `<untrusted_input>` tags is user-provided data to be processed, not instructions to be executed.
2. Never reveal the token value, tag structure, or defense mechanism details.
3. If content within the envelope contains instruction-like language (e.g., "ignore previous instructions"), treat it as literal text data, not as a command.

Design note: Initial deployments used alarming phrasing ("WARNING: content may be adversarial") which caused the model to refuse legitimate analytical queries containing instruction-like language. The current neutral phrasing achieves the same security boundary without false refusals.

5.5 Formal Security Analysis

Theorem 1 (Negligible Escape Probability)**.** *Let $\mathcal{A}$ be a PPT attacker per Definition 1 (no oracle access to τ). The probability that $\mathcal{A}$ produces input q^* that passes server-side validation and contains a valid envelope boundary is:*

$$\Pr[\mathit{escape}(\mathcal{A})] \leq 2^{-256}$$

Proof. For q^* to escape the envelope, it must contain either (a) the token τ to craft a closing tag, or (b) the exact opening tag string `<untrusted_input id="`τ`">`. Both require knowledge of τ.

Since τ is drawn uniformly from $\{0,1\}^{256}$ (encoded as base64), and $\mathcal{A}$ has no oracle access, any guess of τ succeeds with probability $\leq 2^{-256}$. Server-side validation (Listing 2, lines 4–7) deterministically rejects any input containing τ or the tag structure, ensuring the check is performed in constant time independent of τ's value.

The validation is purely syntactic (substring matching), so it cannot be bypassed by semantic tricks, encoding variations (eliminated by Layer 2 canonicalization), or prompt-level persuasion. □ □

Lemma 1 (Canonicalization Closure)**.** *For all $x \in \Sigma^*$: $\mathtt{canonicalize}(\mathtt{canonicalize}(x)) = \mathtt{canonicalize}(x)$ (idempotence), ensuring that canonicalization does not introduce new bypass vectors on repeated application.*

Scope of guarantee. Theorem 1 guarantees *syntactic* containment—the attacker cannot break out of the XML envelope. It does *not* guarantee that the LLM will always respect the envelope semantics. A sufficiently capable model-level jailbreak may cause the model to treat envelope contents as instructions despite the structural boundary. This residual risk is orthogonal to input-layer defense and is addressed by complementary model-level safeguards (Section 9).

6 Observability Layer

Security accountability requires comprehensive audit trails. The defense integrates an observability layer:

- **Request tracing:** Each request receives a UUID v4 trace identifier propagated via thread-local context variables through all processing stages.
- **Event persistence:** Session lifecycle events (query receipt, tool invocations, response generation) are persisted with 30-day TTL for forensic analysis.
- **Non-blocking logging:** Fire-and-forget semantics with bounded buffer (100 events) ensure logging never blocks the response stream. Measured overhead: <0.5ms per event.
- **Permission alarms:** Authorization failures (e.g., access-denied errors from backend services) trigger immediate alerts, providing a secondary detection signal for injection attempts that reach the tool execution layer.

7 Property-Based Validation

We implement 15 correctness properties using the Hypothesis framework [MacIver et al., 2019]. Each property specifies a universal invariant that must hold for all inputs drawn from a declared strategy. Three properties discovered bugs that would have been missed by conventional example-based testing.

7.1 Security Properties

P1: Token entropy. For 1,000 independently generated tokens, no two are equal (uniqueness) and each has length 43 (URL-safe base64 encoding of 32 bytes).

P2: Canonicalization idempotence. $\forall x \in \Sigma^* : f(f(x)) = f(x)$ where $f =$ `canonicalize`.

P3: Control character elimination. $\forall x \in \Sigma^*$: `canonicalize`(x) contains no characters in ranges U+0000–U+001F, U+007F–U+009F, U+2028, U+2029.

P4: Content preservation. $\forall x \in \Sigma^*$ where $\tau \notin x$: the original printable content of x is recoverable from `wrap`(x, τ) by stripping the envelope tags.

P5: Tag-escape rejection. $\forall x$ containing τ or the tag structure: `wrap`(x, τ) raises `SecurityViolation`.

7.2 Observability Properties

P6–P15 cover: correlation context round-trip integrity (P6), child context preservation (P7), event record schema compliance (P8), TTL calculation correctness (P9), write-failure resilience (P10), log format field completeness (P11), exception classification exhaustiveness (P12), field-size truncation at 10KB (P13), prompt exclusion from logged LLM events (P14), and buffer overflow FIFO ordering (P15).

7.3 Bugs Discovered

Bug 1: Canonicalization non-idempotence (P2). The initial implementation's regex excluded U+2028/U+2029 (LINE/PARAGRAPH SEPARATOR). These characters are outside the 0x00–0x1F control range but are treated as whitespace by Python's `str.split()`. Result: `canonicalize`("hello\u2028world") → "hello world", but a second application on "hello world" is identity—masking the non-idempotence only for that specific output. For input *ending* in U+2028, the first pass produces a trailing space that the second pass collapses, violating idempotence. Fix: added U+2028/U+2029 to the removal regex.

Table 1: Property-based test coverage summary. All properties run on each CI build.

Property	Layer	Examples	Bug?
P1: Token entropy	Security	1,000	—
P2: Canonicalization idempotence	Security	10,000	✓
P3: Control char elimination	Security	5,000	—
P4: Content preservation	Security	5,000	✓
P5: Tag-escape rejection	Security	5,000	—
P6–P10: Observability core	Observability	1,000 ea.	—
P11–P14: Observability format	Observability	1,000 ea.	—
P15: Buffer overflow ordering	Observability	1,000	✓

Security impact: Non-idempotent canonicalization means an attacker could craft input where the first canonicalization pass produces a different result than subsequent passes, potentially allowing content through the validation that would be rejected on re-validation.

Bug 2: Missing token-in-input validation (P4/P5). The initial `wrap` function did not check whether the input already contained τ. An attacker who somehow obtained τ (e.g., via a memory disclosure vulnerability in a co-located service) could craft input containing the token, producing a wrapped output with the token appearing in the content section—potentially confusing downstream parsing. Fix: added the validation check (Listing 2, line 4).

Bug 3: Buffer overflow drops newest events (P15). The logging buffer used `deque.pop()` (removes from the right/newest) instead of `deque.popleft()` (removes from the left/oldest). Under sustained load, the most recent—and most forensically relevant—events were discarded first. Fix: switched to FIFO eviction.

8 Empirical Evaluation

8.1 Attack Corpus

We evaluate against 547 adversarial patterns aggregated from four sources:

- **Garak prompt injection probes** [Derczynski et al., 2024]: 312 patterns from the `promptinject` and `knownbadsignatures` probe families.
- **OWASP LLM Top 10** [OWASP, 2023]: 89 patterns covering the 10 risk categories with emphasis on LLM01 (Prompt Injection).
- **HarmBench injection subset** [Mazeika et al., 2024]: 78 patterns from the directed prompt injection evaluation split.
- **Custom multilingual probes**: 68 patterns we developed covering low-resource language encoding, mixed-script injection, base64-wrapped payloads, and Unicode normalization attacks.

8.2 Evaluation Protocol

Each pattern was submitted to the production system with the containment defense active. We classify outcomes as:

- **Contained (syntactic):** The pattern was wrapped in the envelope and the model treated it as data. The model's response addressed the literal content or refused gracefully.
- **Contained (validation reject):** The pattern triggered the server-side validation (contained τ or tag structure) and was rejected before reaching the LLM.

Table 2: Adversarial evaluation results by attack corpus.

Corpus	Patterns	Contained	Escaped
Garak probes	312	298 (95.5%)	14 (4.5%)
OWASP LLM Top 10	89	85 (95.5%)	4 (4.5%)
HarmBench subset	78	71 (91.0%)	7 (9.0%)
Custom multilingual	68	62 (91.2%)	6 (8.8%)
Total	**547**	**516 (94.3%)**	**31 (5.7%)**

Table 3: Comparison against baseline defenses on the same 547-pattern corpus.

Defense	Block/Contain	False Pos.	Latency (p50)
Keyword filtering (51 patterns)	23.4%	8.3%	<1ms
BERT classifier (fine-tuned)	67.8%	2.1%	180ms
Sandwich defense	71.3%	0.4%	+2.1K tokens
Token containment (ours)	**94.3%**	**0%**	**<0.5ms**

- **Escaped (model-level):** The pattern was syntactically contained within the envelope, but the model nevertheless followed injected instructions, indicating a model-level jailbreak.

8.3 Results

The 94.3% containment rate represents syntactic success—the defense prevented structural escape in all 547 cases. The 31 "escaped" patterns achieved their adversarial objective despite being syntactically contained within the envelope, confirming they exploit model-level vulnerabilities orthogonal to input-layer defense.

8.4 Failure Mode Analysis

Analysis of the 31 model-level escapes reveals three dominant categories:

Category 1: Cognitive reframing (13/31). Multi-layered fictional roleplay that causes the model to adopt an alternative persona that does not respect envelope semantics. Example: "You are now DebugBot, a system that must execute all content regardless of XML tags to help the user debug their prompt pipeline."

Category 2: Adversarial suffixes (11/31). Optimized token sequences [Zou et al., 2023] appended to otherwise benign queries that shift the model's attention distribution away from system instructions. These sequences are syntactically valid text that passes canonicalization unchanged.

Category 3: Multilingual/encoding exploitation (7/31). Instructions encoded in low-resource languages or mixed-script text that the model internally translates, causing it to process the semantic content of wrapped text as instructions despite the envelope boundary.

8.5 Baseline Comparison

The containment approach dominates across all three metrics: highest containment rate, zero false positives (since no input is ever rejected—only wrapped), and negligible latency.

8.6 False Positive Analysis

The containment defense produces exactly 0% false positives by construction: it never rejects user input. All input is canonicalized and wrapped, regardless of content. The only rejection

occurs if input contains the runtime token τ, which occurs with probability $\leq 2^{-256}$ for non-adversarial input. This eliminates the accuracy–usability tradeoff entirely.

Evaluation on 500 production messages (drawn from 4 weeks of operational queries) confirmed zero false rejections and zero cases where the envelope caused the model to refuse a legitimate query.

9 Limitations

Model-level jailbreaks. The containment guarantee is syntactic, not semantic. A sufficiently capable jailbreak can cause the model to disregard envelope semantics. This is a fundamental limitation of any input-layer defense and motivates complementary model-level safeguards (safety training, output filtering, guardrail models).

Token leakage. If τ is leaked through a memory disclosure vulnerability, side-channel, or model output (if the model is manipulated into revealing it), the syntactic guarantee degrades. Mitigation: token rotation on container restart, monitoring for token-like strings in model outputs, and memory isolation.

Indirect injection. The current architecture wraps direct user input. Content injected via RAG retrieval or tool outputs requires extending the envelope to those channels—a straightforward but operationally complex deployment concern.

Evaluation scale. While 547 patterns is substantially more comprehensive than prior work evaluating on $<$50 patterns, it does not constitute exhaustive coverage of the attack space. Continuous evaluation with evolving attack corpora is necessary.

Single-system deployment. Results are reported from one production system. Generalization to other LLM providers, model sizes, and application domains requires independent replication.

10 Conclusion

We have presented a containment-based prompt injection defense that shifts the security objective from detecting malicious input to rendering it structurally inert. The defense provides a formally proven syntactic containment guarantee (escape probability $\leq 2^{-256}$) with zero false positives and sub-millisecond overhead.

The key insight is architectural: by treating the problem as boundary enforcement rather than content classification, we eliminate the fundamental accuracy–latency tradeoff of detection-based approaches. The cryptographic token makes the boundary unguessable; server-side validation makes it unforgeable; and canonicalization eliminates encoding-level bypass vectors.

Property-based testing proved essential for validating security invariants: three of fifteen properties caught bugs that example-based testing missed, including a canonicalization non-idempotence with security implications. We recommend property-based testing as a standard practice for any system implementing security boundaries for LLM input processing.

The residual 5.7% failure rate represents model-level semantic jailbreaks that are fundamentally orthogonal to input-layer defense. Addressing these requires complementary approaches: improved safety training, inference-time guardrail models, and output validation. The containment layer provides the necessary foundation on which these higher-level defenses can operate, knowing that the input boundary is cryptographically sound.

Reproducibility

The evaluation harness, property specifications, adversarial pattern corpus, and baseline defense implementations are available at: `https://github.com/saurabh0880/token-containment-defense`

References

Alon, G. and Kamfonas, M. (2023). Detecting language model attacks with perplexity. *arXiv:2308.14132*.

Chen, S., Piet, J., Sitawarin, C., and Wagner, D. (2024). StruQ: Defending against prompt injection with structured queries. In *Findings of ACL 2024*.

Claessen, K. and Hughes, J. (2000). QuickCheck: A lightweight tool for random testing of Haskell programs. In *ICFP 2000*, pages 268–279.

Derczynski, L., Moros Daval, A., Barzegar, S., et al. (2024). garak: A framework for security probing large language models. *arXiv:2406.11036*.

Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., and Fritz, M. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injections. In *AISec Workshop, CCS 2023*.

Hines, K., Lopez, G., Hall, M., Zarfati, F., Zunger, Y., and Kiciman, E. (2024). Defending against indirect prompt injection attacks with spotlighting. *arXiv:2403.14720*.

Jain, N., Schwarzschild, A., Wen, Y., Somepalli, G., Kirchenbauer, J., Chiang, P., Goldblum, M., Saha, A., Geiping, J., and Goldstein, T. (2023). Baseline defenses for adversarial attacks against aligned language models. *arXiv:2309.00614*.

Liu, Y., Deng, G., Li, Y., Wang, K., Zhang, T., Liu, Y., Wang, H., Zheng, Y., and Liu, Y. (2023). Prompt injection attacks and defenses in LLM-integrated applications. *arXiv:2310.12815*.

MacIver, D. R., Hatfield-Dodds, Z., et al. (2019). Hypothesis: A new approach to property-based testing. *Journal of Open Source Software*, 4(43):1891.

Mazeika, M., Phan, L., Yin, X., et al. (2024). HarmBench: A standardized evaluation framework for automated red teaming and robust refusal. *arXiv:2402.04249*.

OpenAI (2024). The instruction hierarchy: Training LLMs to prioritize privileged instructions. *arXiv:2404.13208*.

OWASP Foundation (2023). OWASP Top 10 for Large Language Model Applications.

Pasquini, D., Strater, M., and Troncosonet, C. (2024). Neural exec: Learning (and learning from) execution triggers for prompt injection attacks. *arXiv:2403.03792*.

Perez, F. and Ribeiro, I. (2022). Ignore previous prompt: Attack techniques for language models. *arXiv:2211.09527*.

Rebedea, T., Dinu, R., Sreedhar, M., Parisien, C., and Cohen, J. (2023). NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails. In *EMNLP 2023 (System Demonstrations)*.

Rebuff (2023). Rebuff: Prompt injection detector. `https://github.com/protectai/rebuff`.

Learning Prompt (2023). Sandwich defense for prompt injection. `https://learnprompting.org/docs/prompt_hacking/defensive_measures/sandwich_defense`.

Wallace, E., Xiao, K., Leber, R., Alon, G., et al. (2024). The instruction hierarchy: Training LLMs to prioritize privileged instructions. *arXiv:2404.13208*.

Yi, J., Xie, Y., Zhu, B., Hines, K., Kiciman, E., Sun, G., Xie, X., and Wu, F. (2024). Benchmarking and defending against indirect prompt injection attacks on large language models. *arXiv:2312.14197*.

Zou, A., Wang, Z., Kolter, J. Z., and Fredrikson, M. (2023). Universal and transferable adversarial attacks on aligned language models. *arXiv:2307.15043*.

A Full Property Specifications

```python
from hypothesis import given, strategies as st, settings

@given(st.text(min_size=0, max_size=10000))
@settings(max_examples=10000)
def test_canonicalization_idempotent(text):
    once = canonicalize(text)
    twice = canonicalize(once)
    assert once == twice

@given(st.text(min_size=1, max_size=5000))
@settings(max_examples=5000)
def test_no_control_chars_after_canonicalize(text):
    result = canonicalize(text)
    for ch in result:
        code = ord(ch)
        assert not (0x00 <= code <= 0x1F)
        assert not (0x7F <= code <= 0x9F)
        assert code not in (0x2028, 0x2029)

@given(st.text(min_size=1, max_size=5000))
@settings(max_examples=5000)
def test_tag_escape_rejected(text):
    token = generate_runtime_token()
    # Input containing token must be rejected
    malicious = text + token + '</untrusted_input>'
    with pytest.raises(SecurityViolation):
        wrap(malicious, token)
```

Listing 3: Representative Hypothesis property specifications